

Universidad del Valle de Guatemala

Facultad de Ingeniería

Búsqueda y Evaluación en Ajedrez

Un Análisis Empírico de Técnicas Modernas

Proyecto Final

Inteligencia Artificial – CC3045

José Antonio Mérida Castejón

24 de mayo de 2026

Introducción

El proyecto consistió en desarrollar un motor de ajedrez funcional con el doble propósito de aprender sobre la arquitectura de motores modernos y analizar empíricamente el impacto de cada técnica de búsqueda y evaluación. Además de ser un ejercicio de implementación, el objetivo fue entender *por qué* funcionan estas técnicas, cuánto aporta cada una de forma aislada y qué *trade-offs* se implican en la práctica.

Para ello, se implementaron y compararon distintas variantes de búsqueda y evaluación utilizando técnicas de inteligencia artificial. Cada variante se mantiene en una rama (*branch*) independiente del repositorio, permitiendo comparaciones directas mediante partidas automatizadas que cuantifican el efecto de cada cambio sobre la fuerza de juego real. El código desarrollado está disponible públicamente [7].

La infraestructura base fue adaptada del motor de código abierto *Reckless* [1] implementado en Rust. De este se reutilizó la representación de tablero mediante bitboards, la generación de movimientos legales, el manejo del protocolo UCI y la tabla de transposiciones. La evaluación neuronal original fue eliminada y reemplazada por las técnicas descritas a continuación, con el fin de construir el motor desde sus componentes más fundamentales y comprender cada capa antes de añadir la siguiente.

Índice

1. Descripción del Problema	4
2. Propuesta de la Solución	4
3. Análisis	5
3.1. El Ajedrez como Dominio de IA	5
3.2. Evaluación Heurística Manual (<i>Hand-Crafted Evaluation</i> , HCE)	5
3.3. Redes Neuronales para Evaluación de Posiciones	6
3.4. NNUE: <i>Efficiently Updatable Neural Networks</i>	6
3.4.1. Representación de entrada (<i>feature set</i>)	7
3.4.2. Arquitectura de la red	7
3.4.3. Cuantización e instrucciones SIMD	8
3.5. Destilación de Conocimiento (<i>Knowledge Distillation</i>)	8
3.6. Proceso de Entrenamiento de una NNUE	8
3.6.1. Datasets	8
3.6.2. Ciclo de evaluación y selección	9
3.6.3. Ajuste fino (<i>fine-tuning</i>) y <i>nudging</i>	9
4. Descripción de la Solución	10
4.1. Infraestructura Común	10
4.2. v1-base – Alpha-Beta Básico sin Ordenamiento	10
4.3. v1.5-move-ordering – Ordenamiento de Movimientos	10
4.4. v2-tapered-eval – Evaluación Graduada (PeSTO)	11
4.5. v2.1-nmp – Poda de Movimiento Nulo	11
4.6. v2.2-lmr – Reducción de Movimientos Tardíos	11
4.7. v2.3-rfp – Poda de Futilidad Inversa	11
4.8. v2.4-additional-heuristics – LMP y Extensiones Singulares	12
4.9. v3-nnue – Evaluación Neuronal (NNUE)	12
4.9.1. Progresión de Arquitecturas	12
4.9.2. Experimentos con Protocolos de Entrenamiento	12
4.9.3. Conclusiones del Entrenamiento	13
4.10. v4-nnue-heuristics – Refinamiento de Búsqueda sobre NNUE	13
5. Herramientas Aplicadas	14
5.1. Rust	14
5.2. Reckless [1]	15
5.3. Stockfish [2]	15
5.4. fastchess [3]	16
5.5. En-Croissant [4]	16
5.6. NNUE-PyTorch [5]	16
5.7. Bullet [6]	16
5.8. Claude (Anthropic)	17
6. Resultados	19
6.1. Comparación entre versiones	19
6.2. Análisis de resultados	19
6.2.1. Ordenamiento de movimientos	19

6.2.2.	Evaluación graduada y heurísticas adicionales	20
6.2.3.	Transición a NNUE	20
6.2.4.	Refinamiento de búsqueda sobre NNUE	20
6.2.5.	Contexto de los ELO estimados	21
6.3.	Partidas contra oponente humano	22
6.3.1.	v1.5-move-ordering	22
6.3.2.	v2.4-additional-heuristics	22
7.	Conclusión	22
7.1.	Metodología incremental	22
7.2.	Evaluación heurística	23
7.3.	Ordenamiento de movimientos y heurísticas de búsqueda	23
7.4.	NNUE	23
7.5.	Trabajo futuro	24

1. Descripción del Problema

El ajedrez es un juego de suma cero, información perfecta y complejidad combinatoria alta. El factor de ramificación promedio es de aproximadamente 35 movimientos por posición, lo que hace inviable la búsqueda exhaustiva del árbol de juego completo. El problema central es, dado un estado del tablero, determinar el mejor movimiento posible dentro de un límite de tiempo acotado.

Esto requiere resolver tres subproblemas:

1. **Representación eficiente del estado:** la estructura de datos debe permitir generar y deshacer movimientos legales en tiempo constante o casi constante.
2. **Búsqueda:** explorar el árbol de juego de forma inteligente, podando ramas que no pueden influir en el resultado sin perder exactitud.
3. **Evaluación:** asignar una puntuación numérica a posiciones no terminales que aproxime el resultado real de la partida.

El objetivo del proyecto es comprender cómo los motores modernos abordan cada uno de estos subproblemas, estudiando la teoría detrás de cada técnica y sus fundamentos. A partir de eso, implementar de forma incremental cada algoritmo para medir empíricamente su impacto en la fuerza de juego resultante y analizar los compromisos entre complejidad de implementación y ganancia práctica.

2. Propuesta de la Solución

La metodología consistió en construir el motor como una progresión incremental de versiones, donde cada versión introduce una o más técnicas nuevas sobre la anterior. Se buscó mantener los cambios entre versiones mínimas, sin embargo algunas de las técnicas tienen efectos mínimos que son difíciles de estudiar bajo las restricciones de tiempo y cómputo del proyecto.

Para cada técnica, se siguió un proceso de estudio similar. Primero, se investigó la motivación teórica y como se aplica en motores modernos. Luego, se implementó y evaluó su impacto empírico contra la versión anterior.

Las versiones implementadas son:

1. **v1-base:** alpha-beta con evaluación material, *iterative deepening*, tablas pieza-casilla y *Quiescence Search* integrado desde el inicio para mitigar el efecto horizonte al seguir evaluando capturas ruidosas en nodos hoja.
2. **v1.5-move-ordering:** ordenamiento de movimientos, permitiendo explorar movimientos más *escandalosos* primero.
3. **v2-tapered-eval:** evaluación graduada PeSTO, mantiene dos tablas distintas para el *middle-game* y el *end-game* permitiendo identificar ventajas y desventajas en diferentes fases del juego.
4. **v2.1-nmp:** poda de movimiento nulo (*Null Move Pruning*), consiste en *regalar* un movimiento adicional al oponente. Si la posición sigue teniendo una buena evaluación, entonces no debemos revisar las siguientes posiciones.

5. **v2.2-lmr**: reducción de movimientos tardíos (*Late Move Reductions*), consiste en penalizar los movimientos silenciosos al final de la búsqueda mediante una disminución de la profundidad de búsqueda. Es una extensión de *move ordering*.
6. **v2.3-rfp**: poda de futilidad inversa (*Reverse Futility Pruning*), consiste en establecer un *threshold* de evaluación estática donde esta es tan buena que nos garantiza que la posición actual es buena y evitamos la búsqueda de más movimientos.
7. **v2.4-additional-heuristics**: optimización de LMR y extensiones singulares. Se extiende *Late Move Reduction* permitiéndole realizar podas más agresivas y se implementan extensiones singulares (*Singular Extensions*) para identificar jugadas forzadas únicas, extendiendo la profundidad de búsqueda en líneas críticas.
8. **v3-nnue**: reemplazo de la evaluación heurística por una red neuronal NNUE.
9. **v4-nnue-heuristics**: ajuste de parámetros de heurísticas para ajustarse a una evaluación más robusta, implementación de ventanas de aspiración (*Aspiration Windows*). Esta heurística consiste en proponer un rango (α, β) al iniciar la búsqueda.

La métrica principal de comparación es el ELO estimado mediante partidas contra Stockfish a distintos niveles de rating y demás versiones del motor, utilizando la herramienta *fastchess* [3] para la automatización de los torneos. Esta métrica permite no solo medir la fuerza absoluta de cada versión, sino también cuantificar cuánto mejoró cada versión individualmente. Además, nos permite tener un *head to head* directo entre diferentes versiones para poder determinar superioridad.

Adicionalmente, se realizaron pruebas con un voluntario humano (estimado ≈ 2200 de ELO) para analizar cualitativamente las tendencias, fortalezas y debilidades de cada versión principal del motor desde una perspectiva humana.

3. Análisis

Esta sección presenta el análisis del problema y el dominio de solución. La investigación teórica realizada para comprender qué técnicas existen, por qué funcionan y cuáles son sus limitaciones. El desarrollo de un motor de ajedrez involucra terminología específica del dominio del ajedrez competitivo, de la teoría de búsqueda en árboles y del aprendizaje automático. Se definen estos conceptos como base para las decisiones de diseño descritas en secciones posteriores.

3.1. El Ajedrez como Dominio de IA

El ajedrez ha sido desde sus inicios uno de los problemas de referencia de la inteligencia artificial. Su atractivo como *benchmark* se basa en que las reglas son completamente conocidas y deterministas (información perfecta, sin azar), pero la complejidad combinatoria del árbol de juego hace imposible la búsqueda exhaustiva. Se estima que el número de posiciones de ajedrez legales es del orden de 10^{44} (*número de Shannon*), y el número de partidas posibles supera 10^{120} .

3.2. Evaluación Heurística Manual (*Hand-Crafted Evaluation*, HCE)

Durante décadas, la evaluación de posiciones en motores de ajedrez se realizó mediante funciones heurísticas diseñadas manualmente por expertos humanos, conocidas como *Hand-Crafted Evaluation* (HCE). La idea central es asignar un puntaje numérico a cada posición que refleje la ventaja del jugador activo, sin necesidad de buscar más profundo en el árbol. [8]

Una HCE típica combina múltiples términos, acumulados a lo largo de décadas de conocimiento ajedrecístico:

- **Balance material:** cada tipo de pieza tiene un valor en centipeones (un centipeón equivale a 1/100 de peón). Los valores convencionales son aproximadamente: Peón 100, Caballo 320, Alfil 330, Torre 500, Dama 900. La diferencia de material entre los dos jugadores es el término más influyente.
- **Tablas pieza-casilla (*Piece-Square Tables*, PST):** bonificaciones o penalizaciones por la casilla que ocupa cada pieza. Por ejemplo, los caballos son más fuertes en el centro, los peones avanzados valen más en el *endgame*, y el rey debe estar protegido en el *middlegame* pero activo en el *endgame*.
- **Estructura de peones:** peones doblados, aislados, pasados, retrasados. Un peón pasado (sin peones rivales que lo bloqueen en columnas adyacentes) puede ser decisivo en finales. Los peones doblados (dos peones del mismo color en la misma columna) son generalmente una debilidad.
- **Movilidad:** el número de movimientos disponibles para cada pieza es un indicador de su actividad. Piezas con poca movilidad están mal colocadas.
- **Seguridad del rey:** cobertura de peones alrededor del rey, número de atacantes cerca, presencia de piezas enemigas en columnas abiertas hacia el rey.
- **Par de alfiles:** poseer ambos alfiles cuando el rival no los tiene es una ventaja posicional reconocida, especialmente en posiciones abiertas.
- **Torres en columnas abiertas o semi-abiertas:** una torre en una columna sin peones propios es más activa y puede ejercer presión sobre el rival.

Las HCE fueron el estado del arte durante décadas. Algunos motores como Stockfish (en sus versiones pre-NNUE), Komodo y Houdini alcanzaron niveles sobrehumanos basándose exclusivamente en búsqueda alpha-beta con evaluación heurística. Sin embargo, diseñar y ajustar una HCE es un proceso laborioso que requiere conocimiento avanzado del dominio. Cada término requiere intuición ajedrecística y ajuste manual de pesos, donde los términos interactúan entre sí de formas difíciles de predecir.

3.3. Redes Neuronales para Evaluación de Posiciones

La evaluación de posiciones de ajedrez puede modelarse como un problema de regresión. Dada una representación del tablero, predecir un escalar que refleje la ventaja del jugador activo. Las redes neuronales son candidatos naturales para esta tarea, ya que pueden aprender patrones complejos a partir de grandes volúmenes de datos sin necesidad de *feature engineering* manual.

Sin embargo, la aplicación directa de arquitecturas profundas en un motor de ajedrez presenta un problema práctico fundamental en cuanto a la velocidad de inferencia. Un motor típico evalúa entre 10^5 y 10^7 posiciones por segundo, si cada evaluación requiere una pasada completa por una red profunda el motor se vuelve demasiado lento para ser competitivo.

3.4. NNUE: *Efficiently Updatable Neural Networks*

La arquitectura NNUE, propuesta por Yu Nasu en 2018 y popularizada por su integración en Stockfish a partir de 2020, resuelve el problema de velocidad mediante dos principios:

1. **Arquitectura superficial:** la red es deliberadamente pequeña (típicamente una capa oculta de pocas centenas de neuronas), lo que hace la inferencia barata.
2. **Actualización incremental del acumulador:** la capa de entrada representa *features* de la posición que solo cambian en un pequeño subconjunto al realizar un movimiento. En lugar de recomputar la activación completa de la primera capa oculta, se mantiene un acumulador que se actualiza de forma incremental. Solo se suman o restan las columnas de los pesos correspondientes a las características que cambiaron. Esto reduce el costo de la primera capa de $O(n)$ a $O(\Delta n)$, donde $\Delta n \ll n$. [9]

3.4.1. Representación de entrada (*feature set*)

La elección del *feature set* determina qué información recibe la red y cuántas entradas tiene la primera capa. Los *feature sets* más sofisticados utilizados por Stockfish son *HalfKP* y su sucesor *HalfKA*:

- **HalfKP:** para cada pieza p en casilla s , genera una característica distinta por cada posición posible del rey propio. Esto permite a la red razonar sobre la relación espacial entre cada pieza y el rey, capturando conceptos como “esta torre está protegiendo a mi rey” sin que eso esté programado explícitamente. El resultado es un vector de entrada de $\sim 41,024$ dimensiones por perspectiva. [10]
- **HalfKA (y HalfKA^{v2}):** extiende HalfKP incluyendo también la posición del rey como una característica adicional, y aplica la relativización a ambos reyes. Esto incrementa la capacidad expresiva de la red pero también su dimensionalidad de entrada y, por tanto, el costo de actualización del acumulador. [10]

Estas arquitecturas son notablemente complejas de entrenar. Al condicionar cada característica a la posición del rey, el espacio de entradas es mucho más disperso y la red necesita haber visto suficientes ejemplos de cada configuración de rey para generalizar bien. Stockfish entrena sus redes con conjuntos de datos del orden de **cientos de miles de millones de posiciones**, generadas por el propio Stockfish jugando contra sí mismo y re-anotadas de forma iterativa, un proceso que requiere semanas de cómputo distribuido en hardware de alto rendimiento. Replicar esto a escala queda completamente fuera del alcance de un proyecto individual.

En este proyecto se utilizó una representación más simple de 768 características: 12 tipos de pieza ($6 \text{ piezas} \times 2 \text{ colores}$) $\times 64$ casillas, una entrada binaria por cada combinación (pieza, casilla) presente en el tablero. No relativiza al rey y no captura relaciones espaciales implícitas, pero es suficiente para validar la arquitectura y entrenable con un *dataset* de tamaño razonable. Además, usualmente se recomienda iniciar con arquitecturas más simples antes de intentar implementar *HalfKA* o *HalfKP*. [11]

3.4.2. Arquitectura de la red

La red sigue el esquema *perspective network*: se mantienen dos acumuladores paralelos (uno por perspectiva), que se concatenan antes de las capas posteriores. La función de activación utilizada es *Square Clipped ReLU* (SReLU), que acota las activaciones en $[0, 1]$, lo que simplifica la cuantización a enteros de 8 o 16 bits para inferencia eficiente con instrucciones SIMD.

La arquitectura general es:

$$(768 \rightarrow H) \times 2 \xrightarrow{\text{concat}} 2H \rightarrow \dots \rightarrow 1$$

donde H es el tamaño de la capa oculta (384, 512 o 768 en las variantes exploradas).

3.4.3. Cuantización e instrucciones SIMD

Para alcanzar velocidades de inferencia comparables a la evaluación heurística, los pesos de la red se cuantizan a enteros de 16 bits (capa de entrada) y 8 bits (capas posteriores) [9]. Las operaciones de punto fijo se ejecutan mediante instrucciones SIMD (AVX2/AVX-512 en x86_64), que procesan 16–32 enteros en paralelo por ciclo de reloj. Rust permite acceso directo a estas instrucciones mediante *intrinsics* o bibliotecas como `std::simd`, garantizando rendimiento predecible sin pasar por un *garbage collector*.

3.5. Destilación de Conocimiento (*Knowledge Distillation*)

El entrenamiento de la NNUE no se realiza directamente sobre resultados de partidas (ganó/perdió/empató), sino mediante *knowledge distillation*. Se utiliza un motor de referencia para anotar un conjunto masivo de posiciones con evaluaciones numéricas en centipeones. La red aprende a imitar estas evaluaciones.

La función de pérdida típica combina el error cuadrático medio sobre las evaluaciones del motor de referencia con el resultado real de la partida, ponderados por un hiperparámetro λ [11]:

$$\mathcal{L} = \lambda \cdot \text{MSE}(\hat{y}, y_{\text{SF}}) + (1 - \lambda) \cdot \text{MSE}(\hat{y}, y_{\text{resultado}})$$

3.6. Proceso de Entrenamiento de una NNUE

El entrenamiento de una NNUE es un ciclo iterativo que combina generación u obtención de datos, optimización y evaluación empírica. A diferencia del entrenamiento de redes convencionales, donde el criterio de calidad está bien definido (precisión en un conjunto de prueba), en motores de ajedrez la métrica real es la fuerza de juego en torneos, lo que introduce una capa adicional de complejidad. Además, acorde a la Wiki de Stockfish [12] la calidad del dataset y los hiperparámetros a utilizar se determinan completamente de manera empírica. Citando también un ciclo de entrenamiento de la red de Stockfish [13], no se busca necesariamente probar la red luego de terminar algún ciclo de entrenamiento o con menores valores de pérdida. El objetivo principal es generar diferentes versiones para poder compararlas una con la otra y examinar el efecto en el nivel de juego del engine entero.

3.6.1. Datasets

El primer paso es construir u obtener un conjunto masivo de posiciones anotadas. Para fines de este proyecto, se utilizaron dos datasets principales.

- **nodes5000pv_UHO**: Un conjunto de datos “clásico” generado por Stockfish con alrededor de 1.6 mil millones de posiciones evaluadas a una profundidad de 5,000 nodos por movimiento. Normalmente utilizado como base para diferentes NNUEs. [12]
- **test80-mar2023-2tb7p-v6-sk16**: Un conjunto de datos generados por *Lc0* con alrededor de 470 millones de posiciones, un engine que utiliza *MCTS* y aprendizaje por refuerzo [14]. Estos sets de datos normalmente se utilizan como complementarios a una red base ya entrenada, utilizando un *learning rate* más bajo para mover ligeramente los pesos [12].

Los datasets se almacenan en formatos binarios comprimidos (`.binpack`, `.bin`) para hacer viable su procesamiento en memoria durante el entrenamiento. El peso total de ambos datasets usados es de $\approx 65GB$ y sumando demás datasets de prueba se excedieron los $\approx 100GB$ de almacenamiento utilizado.

3.6.2. Ciclo de evaluación y selección

Una vez entrenado un candidato, no basta con medir la pérdida sobre un conjunto de validación: la pérdida baja no garantiza que el motor juegue mejor. El procedimiento estándar consiste en ejecutar torneos automatizados de 100–400 partidas entre el candidato y la *baseline*, promoviendo únicamente los candidatos que demuestran mejora estadísticamente significativa ($LOS > 95\%$).

Sin embargo, ejecutar un torneo completo por cada checkpoint es computacionalmente costoso. Dado el cómputo disponible en este proyecto, se adoptó una estrategia de selección basada en checkpoints empíricos. En lugar de evaluar cada época individualmente, se exportaron y compararon checkpoints específicos del proceso de entrenamiento:

- **Primeras épocas:** la red aprende rápidamente los patrones más básicos (balance material, posicionamiento de piezas). Los checkpoints tempranos sirven como referencia para cuantificar cuánto aporta el entrenamiento adicional.
- **Plateau de pérdida:** punto donde la función de pérdida deja de descender de forma significativa. Suele corresponder a la mejor generalización antes de que el modelo empiece a sobreajustar al ruido del dataset o diverger.
- **Última época:** el modelo completamente entrenado, que puede haber sobreajustado levemente pero aprovecha todos los datos vistos.

Comparar estos checkpoints entre sí, en lugar de ejecutar un gauntlet completo por cada variante de hiperparámetros o dataset, permitió identificar qué punto del entrenamiento producía el mejor motor sin requerir cientos de partidas adicionales por experimento. El ganador de cada comparación pasaba a ser la nueva *baseline* para el siguiente experimento.

3.6.3. Ajuste fino (*fine-tuning*) y *nudging*

Tras el entrenamiento principal, es común realizar una o varias rondas de ajuste fino:

- ***Fine-tuning* con datos de mayor calidad:** si se dispone de posiciones generadas por un motor más fuerte o con mayor profundidad de análisis, se puede continuar el entrenamiento desde los pesos ya aprendidos con esos datos y una tasa de aprendizaje muy baja, evitando destruir lo aprendido pero permitiendo que la red se acerque a evaluaciones de mayor precisión.
- ***Nudging* hacia resultado de partida:** se cambia parcialmente el objetivo de la pérdida hacia la predicción del resultado real (victoria/derrota/tablas) en lugar de la evaluación de Stockfish. Dado que las evaluaciones de Stockfish reflejan la perspectiva de un motor HCE con sus propios sesgos, el *nudging* puede corregir sesgos sistemáticos hacia el comportamiento “real” del ajedrez. Solo es efectivo con tasas de aprendizaje muy bajas y pocas épocas; de lo contrario el modelo olvida lo aprendido previamente (*catastrophic forgetting*).

Nota: Nudging de por si no es el término utilizado formalmente, si no que es un patrón de entrenamiento que observé en algunos ciclos de Stockfish y decidí llamarle de esa manera para mantener control de mis versiones.

4. Descripción de la Solución

4.1. Infraestructura Común

Todas las variantes comparten los siguientes componentes, heredados o adaptados de *Reckless*:

- **Representación del tablero:** bitboards de 64 bits por pieza y color. Permite operaciones de conjuntos sobre casillas en tiempo constante.
- **Generación de movimientos:** generación legal separada en movimientos ruidosos (capturas y promociones) y silenciosos, con soporte para *en passant* y enroques.
- **Búsqueda en profundización iterativa:** se busca a profundidad 1, luego 2, y así sucesivamente hasta agotar el tiempo. El resultado de cada iteración alimenta la siguiente mediante la tabla de transposiciones.
- **Búsqueda alpha-beta (negamax):** poda de ramas que no pueden afectar el resultado, es la base de toda la búsqueda.
- **PVS (*Principal Variation Search*):** el primer movimiento se busca con ventana completa; los demás se buscan primero con ventana nula. Si alguno supera alpha, se hace una nueva búsqueda completa para confirmarlo.
- **Búsqueda de quietud (*quiescence search*):** al llegar a profundidad cero se continúa buscando solo capturas hasta que la posición sea estable, evitando el efecto horizonte.
- **Tabla de transposiciones:** caché indexada por hash Zobrist que almacena resultados de posiciones ya buscadas. Permite evitar búsquedas redundantes en posiciones alcanzadas por distintos órdenes de movimientos.
- **Detección de tablas:** por regla de 50 movimientos, repetición de posición y material insuficiente (R vs R, R+A vs R, R+C vs R, alfiles de mismo color).

4.2. v1-base – Alpha-Beta Básico sin Ordenamiento

La variante más simple. Implementa alpha-beta con profundización iterativa y una función de evaluación heurística basada en:

- **Material:** valores fijos por tipo de pieza (Peón 100, Caballo 320, Alfil 330, Torre 500, Dama 900).
- **Tablas pieza-casilla (PST):** bonificaciones posicionales por casilla según la *Simplified Evaluation Function* de Tomasz Michniewski [15]. Una única tabla por pieza, independiente de la fase del juego.

Los movimientos se generan y prueban en el orden que produce el generador, sin ningún tipo de ordenamiento. Sirve como línea base mínima.

4.3. v1.5-move-ordering – Ordenamiento de Movimientos

Sobre la misma evaluación, se agrega un sistema de ordenamiento de movimientos por etapas (*staged move ordering*):

- **Movimiento de la tabla de transposiciones:** el mejor movimiento de la búsqueda anterior se intenta primero.

- **Capturas buenas (SEE):** las capturas se filtran mediante *Static Exchange Evaluation*, que determina si una secuencia de capturas en una casilla resulta en ganancia material. Las capturas perdedoras se posponen.
- **Movimientos silenciosos con historial:** movimientos no capturantes puntuados por tablas de historial que aprenden qué movimientos generaron cortes beta en búsquedas anteriores. Incluye historial de quietud, historial ruidoso e historial de continuación (pares de movimientos consecutivos).
- **Capturas malas:** las capturas que pierden material según SEE se intentan al final.

El ordenamiento no afecta la exactitud del resultado, solo la eficiencia. Alpha-Beta poda más ramas cuando ve movimientos buenos primero.

4.4. v2-tapered-eval – Evaluación Graduada (PeSTO)

Se reemplaza la función de evaluación con las tablas de *PeSTO* (*Piece-Square Tables Optimization*), que introducen evaluación graduada (*tapered evaluation*) [16]:

- **Dos tablas por pieza:** una para apertura/mediojuego y otra para finales, con valores de material distintos en cada fase.
- **Fase de juego:** calculada sumando el material remanente. A medida que caen piezas, la evaluación transiciona suavemente hacia los valores de final.
- **Interpolación:** $\text{score} = (\text{mg_score} \times \text{fase} + \text{eg_score} \times (24 - \text{fase})) / 24$.

4.5. v2.1-nmp – Poda de Movimiento Nulo

Se agrega *Null Move Pruning* (NMP). Si la posición evaluada es tan buena que incluso cediendo un movimiento gratis al rival este no puede superar beta, se poda sin buscar más. Se implementa pasando el turno (*null move*), buscando a profundidad reducida y si el resultado sigue siendo $\geq \beta$, se retorna beta. La idea central es que *pasar* un turno es una peor decisión que cualquiera de los movimientos legales en la mayoría de posiciones de ajedrez. Se desactiva en jaque, cuando el movimiento anterior también fue nulo, cuando solo quedan peones y rey (riesgo de *zugzwang*), y cuando la evaluación estática no supera beta. [17]

4.6. v2.2-lmr – Reducción de Movimientos Tardíos

Se agrega *Late Move Reductions* (LMR), la intuición es una extensión directa del ordenamiento de movimientos. Si ya ordenamos los movimientos de mejor a peor, los que aparecen al final de la lista probablemente sean malos y no valga la pena explorarlos a profundidad completa. En vez de descartarlos por completo, se buscan a profundidad reducida. Si sorpresivamente resultan ser mejores de lo esperado y superan alpha, se rehace la búsqueda a profundidad completa para confirmar. Normalmente solo se aplica a movimientos silenciosos y cuando la profundidad es ≥ 3 . [18]

4.7. v2.3-rfp – Poda de Futilidad Inversa

Se agrega *Reverse Futility Pruning* (RFP). La idea es simple, si la evaluación estática de la posición actual ya es tan buena que supera beta por un margen amplio ($75 \times \text{profundidad}$), significa que estamos tan arriba que ningún movimiento razonable del rival puede revertir esa ventaja. Es el

equivalente a dejar de calcular cuando ya se está ganando una dama de material. Se retorna la evaluación estática sin buscar más. Solo se aplica en nodos fuera de jaque y a profundidad ≤ 8 . [19]

4.8. v2.4-additional-heuristics – LMP y Extensiones Singulares

Se agregan dos técnicas adicionales:

- **Poda de Movimientos Tardíos (LMP):** versión más agresiva de LMR para nodos superficiales. A profundidad ≤ 4 , si ya se intentaron suficientes movimientos silenciosos sin encontrar nada mejor, los restantes se omiten completamente en vez de reducirse. La lógica es la misma que LMR pero llevada al extremo. Si los primeros N movimientos no mejoraron α , los siguientes probablemente tampoco lo harán.
- **Extensiones Singulares:** si un movimiento es el único que evita un resultado significativamente peor en una posición, es probablemente forzado y vale la pena buscarlo más profundo. Se detecta buscando todos los demás movimientos a media profundidad con una ventana justo por debajo del puntaje almacenado en la tabla de transposiciones. Si ninguno supera ese umbral, el movimiento es “singular” y recibe 1 capa de profundidad adicional de búsqueda, permitiendo al motor profundizar en líneas tácticas críticas que de otro modo quedarían en el horizonte.

4.9. v3-nnue – Evaluación Neuronal (NNUE)

Esta etapa reemplaza la función de evaluación heurística por una NNUE. A diferencia de las versiones anteriores, donde cada cambio tenía un efecto claro y medible de forma casi inmediata el entrenamiento de NNUEs es un proceso fundamentalmente empírico. No existe una fórmula que determine de antemano qué arquitectura, qué combinación de datos o qué régimen de entrenamiento producirá el mejor motor. La práctica estándar en la comunidad de motores de ajedrez es generar candidatos, enfrentarlos entre sí mediante torneos automatizados y quedarse con los ganadores, iterando hasta encontrar una combinación que funcione.

4.9.1. Progresión de Arquitecturas

Se exploró una progresión incremental de tamaños de capa oculta, manteniendo el *feature set* de 768 características en todos los casos. En este caso, se realizaron experimentos con 384, 512, 768 y 1024 neuronas en la capa oculta. Se observaron ganancias cuantificables hasta llegar a 1024, por lo cuál los experimentos notables se restringen a tamaños de capa oculta de 768 y 1024. Adicionalmente, a lo largo del proceso se descartó completamente un aumento de profundidad de la red por recomendación de un experto de la comunidad. [11]

4.9.2. Experimentos con Protocolos de Entrenamiento

Una parte significativa de los experimentos no involucró cambios en la arquitectura sino en la composición del *dataset* y el protocolo de entrenamiento. Los experimentos se compararon mediante checkpoints seleccionados (primeras épocas, plateau de pérdida y última época) en lugar de gauntlets completos, dado el cómputo disponible.

La progresión de experimentos fue la siguiente:

1. **768 vs 1024 neuronas, entrenamiento base (UHO):** la arquitectura de 768 neuronas superó a la de 1024. La hipótesis más probable es que la red más pequeña realiza más evalua-

ciones por segundo, lo que compensa su menor capacidad expresiva con mayor profundidad de búsqueda en el tiempo disponible.

2. ***Nudging* sobre 768 (protocolo inicial):** siguiendo la documentación de Stockfish [12], se intentó un ajuste fino con datos de *lc0* [14] sobre la red de 768 neuronas. El resultado fue equivalente a la *baseline* ($+0 \pm 3$ ELO), sin mejora estadísticamente significativa. En este punto también se observó que la pérdida había alcanzado un plateau, sugiriendo que la red había extraído lo que podía del dataset base.
3. **1024 neuronas con *nudging* (protocolo ajustado):** se repitió el entrenamiento de 1024 neuronas sobre UHO y se aplicó *nudging* con un protocolo más suave que el anterior (tasa de aprendizaje más baja, menos épocas). Esta versión sí superó a la 1024 base, sugiriendo que el protocolo de *nudging* importa tanto como la arquitectura.
4. **1024 con *nudging* vs 768 base:** al enfrentar la mejor red de 1024 contra la *baseline* de 768, la 1024 perdió ELO. La ventaja del *nudging* no fue suficiente para compensar el costo de inferencia de la red más grande.
5. ***Nudging* sobre 768 (protocolo ajustado):** dado que el protocolo más suave había funcionado con 1024, se intentó aplicarlo también sobre 768. El resultado fue negativo. La red perdió ELO respecto a la *baseline*. El mismo protocolo que mejoró 1024 perjudicó a 768, evidenciando que estos ajustes no son transferibles entre arquitecturas.
6. **Mayor variedad de datos:** se intentó un entrenamiento base con una mezcla más amplia de datos (UHO + datos adicionales) para aumentar la diversidad de posiciones vistas. Tampoco produjo mejora sobre la *baseline* de 768.

4.9.3. Conclusiones del Entrenamiento

El conjunto de experimentos ilustra la naturaleza profundamente empírica del entrenamiento de NNUEs. La intuición detrás del *nudging* es sólida, las evaluaciones de Stockfish pueden heredar sesgos posicionales propios y exponer la red brevemente al resultado real de partidas a tasa de aprendizaje muy baja puede corregirlos sin destruir lo aprendido. Sin embargo, en la práctica el protocolo exacto (tasa de aprendizaje, número de épocas, datos usados para el ajuste) interactúa con la arquitectura de formas difíciles de predecir. Lo que funcionó para 1024 no funcionó para 768, y viceversa.

La conclusión principal no es qué protocolo es mejor, sino que no existe un protocolo universalmente superior a esta escala. El progreso en entrenamiento de NNUEs es esencialmente una búsqueda empírica, donde se generan candidatos, se comparan y se mantienen únicamente los ganadores. Esto es exactamente lo que hace Stockfish, pero con órdenes de magnitud más cómputo y una infraestructura de evaluación continua que va completamente más allá del alcance de este proyecto.

4.10. v4-nnue-heuristics – Refinamiento de Búsqueda sobre NNUE

Esta versión mantiene la misma red NNUE de v3 y se enfoca en mejorar la búsqueda. La motivación es que los parámetros de poda calibrados para HCE no son necesariamente óptimos bajo evaluación neuronal. Una NNUE es más precisa y estable que una HCE, lo que permite ajustar los márgenes de poda de forma más agresiva sin riesgo de descartar posiciones buenas. Las técnicas añadidas son:

- **Ventanas de aspiración (*Aspiration Windows*):** en lugar de iniciar cada profundidad con una ventana $[-\infty, +\infty]$, se propone una ventana estrecha $[\text{prev} - 20, \text{prev} + 20]$ centrada en el resultado de la profundidad anterior. Si la búsqueda cae fuera, la ventana se expande y se reintenta. A profundidades altas, donde la puntuación es estable, esto reduce significativamente el trabajo por iteración.
- **Heurística de mejora (*Improving*):** rastrea si la evaluación estática en el nodo actual es mejor que la de hace dos plies (profundidades). Si la posición está deteriorándose, los márgenes de poda se ajustan de forma conservadora; si mejora, se podan más agresivamente. Se aplica directamente a los umbrales de RFP y LMP.
- **Razoring:** a profundidades ≤ 4 , si la evaluación estática está tan por debajo de alpha que una búsqueda completa no puede recuperar la diferencia, se cae directamente en *quiescence search*. Evita búsquedas completas costosas en nodos claramente perdedores.
- **Reducción iterativa interna (IIR):** si no hay movimiento de la tabla de transposiciones a profundidad ≥ 4 , el ordenamiento de movimientos será pobre de todas formas. En vez de buscar a profundidad completa con mal ordenamiento, se reduce en 1 la profundidad.
- **Extensiones de jaque:** los movimientos que dan jaque reciben 1 de profundidad adicional de búsqueda, además de las extensiones singulares. Esto fuerza al motor a analizar más profundamente las secuencias forzadas.
- **Poda de futilidad (*Futility Pruning*):** complementaria a RFP, que poda cuando la evaluación supera beta, esta poda descarta movimientos silenciosos cuando la evaluación estática está suficientemente por debajo de alpha a baja profundidad.
- **Poda SEE en búsqueda:** las capturas que pierden material según la evaluación estática de intercambio (*SEE*) se descartan a profundidad ≤ 6 , en lugar de solo depriorizarse en el ordenamiento.
- **LMR ajustado:** la fórmula de reducción incorpora el historial de movimientos silenciosos y un bono por jaque, haciéndola más selectiva sobre qué movimientos realmente se reducen.

Adicionalmente, se reajustaron los parámetros numéricos de las heurísticas existentes (márgenes de RFP, umbrales de LMP, profundidades de activación) para adaptarse a la mayor precisión de la evaluación NNUE.

5. Herramientas Aplicadas

Esta sección documenta los programas, frameworks y recursos externos utilizados durante el desarrollo del proyecto. Los algoritmos y técnicas de inteligencia artificial aplicados (alpha-beta, heurísticas de búsqueda, NNUE) se explican en profundidad en las secciones de Análisis y Descripción de la Solución; aquí se describen únicamente las herramientas concretas que hicieron posible su implementación, entrenamiento y evaluación.

5.1. Rust

El motor está implementado íntegramente en Rust. La elección del lenguaje responde a requisitos concretos del dominio:

- **Rendimiento:** los motores de ajedrez son aplicaciones de tiempo real donde las pausas de GC son inaceptables. Rust gestiona la memoria de forma estática mediante su sistema de *ownership*, garantizando latencia predecible sin costo en tiempo de ejecución.
- **Abstracciones de costo cero:** iteradores, genéricos y *traits* compilan a código tan eficiente como C, permitiendo estructurar el código de forma legible sin sacrificar rendimiento.
- **Acceso a instrucciones SIMD:** la evaluación NNUE requiere multiplicaciones matriciales sobre enteros de 8/16 bits en vectores anchos. Rust expone *intrinsics* AVX2/AVX-512 directamente, y la biblioteca estándar incluye `std::simd` para operaciones portables. Esto es esencial para mantener la inferencia de la red dentro del presupuesto de tiempo del motor.
- **Concurrencia segura:** la arquitectura *Lazy SMP* (el motor lanza múltiples hilos buscando en paralelo con la misma tabla de transposiciones) requiere compartir estado mutable entre hilos sin condiciones de carrera. El sistema de tipos de Rust garantiza esto en tiempo de compilación.

5.2. Reckless [1]

Reckless es un motor de ajedrez de código abierto escrito en Rust que sirvió como base del proyecto. Se reutilizaron sus componentes de infraestructura:

- Representación de tablero mediante bitboards.
- Generación de movimientos legales (incluyendo *en passant*, enroques y promociones).
- Manejo del protocolo UCI (*Universal Chess Interface*), que permite la comunicación estándar con interfaces gráficas y herramientas de torneo.
- Tabla de transposiciones con *hashing* Zobrist.

La evaluación neuronal original de *Reckless* fue eliminada para implementar las variantes descritas en este informe.

5.3. Stockfish [2]

Stockfish es el motor de ajedrez de código abierto más fuerte del mundo, con un ELO estimado de ~3600+. Opera mediante una búsqueda alpha-beta altamente optimizada combinada con una NNUE propia entrenada sobre cientos de millones de posiciones. En versiones recientes incorpora también redes más profundas.

Se utilizó en dos roles dentro del proyecto:

- **Oponente de referencia:** al configurarse con un *rating* objetivo (mediante el parámetro `UCI_LimitStrength`), permite estimar el ELO del motor en desarrollo de forma controlada y reproducible.
- **Oráculo de evaluación:** en la generación del *dataset* de entrenamiento de la NNUE, se usó Stockfish para anotar posiciones con evaluaciones en centipeones, implementando la destilación de conocimiento descrita en la sección de Análisis.

5.4. fastchess [3]

fastchess es una herramienta de línea de comandos para automatizar torneos entre motores UCI. Implementa el formato de torneo *gauntlet* (un motor retador contra múltiples oponentes) y *round-robin*, con soporte para *opening books* en formato EPD/PGN para garantizar diversidad de posiciones iniciales.

Las métricas reportadas por **fastchess** que se utilizaron para comparar versiones son:

- **ELO**: diferencia de ELO estimada entre los dos motores a partir del resultado del torneo, con intervalo de confianza.
- **nELO** (*normalized ELO*): variante del ELO que normaliza por la tasa de tablas del torneo, más informativo cuando las tablas son frecuentes.
- **LOS** (*Likelihood of Superiority*): probabilidad de que el motor retador sea genuinamente más fuerte, asumiendo que los resultados siguen una distribución trinomial. Un $LOS > 95\%$ se considera resultado estadísticamente significativo.

Cada prueba se ejecutó con control de tiempo de $10 + 0,1$ segundos por partida (*increment*), 1 hilo y 16 MB de *hash*, para mantener condiciones homogéneas entre versiones.

5.5. En-Croissant [4]

En-Croissant es una interfaz gráfica de escritorio para motores UCI. Se utilizó durante el desarrollo para depurar el comportamiento del motor: analizar posiciones concretas, verificar que los movimientos generados eran legales, inspeccionar la línea principal (*principal variation*) reportada por el motor y contrastar evaluaciones contra Stockfish. También se empleó para las partidas de prueba contra el oponente humano.

5.6. NNUE-PyTorch [5]

NNUE-PyTorch es una implementación de referencia del entrenamiento de NNUEs basada en PyTorch, desarrollada y mantenida por la comunidad de Stockfish. Proporciona:

- Un *data loader* optimizado que lee posiciones en formato binario comprimido (**.binpack**), permitiendo procesar cientos de millones de posiciones sin cargarlas todas en memoria.
- Una implementación modular de la arquitectura NNUE que puede adaptarse a distintos *feature sets* y tamaños de red.
- Utilidades para exportar los pesos entrenados al formato binario **.nnue** que los motores consumen en tiempo de ejecución.

Se utilizó como punto de partida para experimentar con la arquitectura, sin embargo esta herramienta no soporta arquitecturas más simples. Por lo tanto, se utilizó únicamente la documentación y ejemplos como referencia antes de migrar el entrenamiento a *Bullet*.

5.7. Bullet [6]

Bullet es un *framework* de entrenamiento de NNUEs escrito en Rust y CUDA, diseñado específicamente para la velocidad de entrenamiento en el contexto de motores de ajedrez. Frente a *NNUE-PyTorch*, ofrece:

- **Mayor rendimiento:** al estar escrito directamente en Rust/CUDA sin pasar por el *overhead* de Python, procesa más posiciones por segundo.
- **Soporte para arquitecturas simples:** Este proyecto utilizó una red relavitamente sencilla, la implementación de feature sets más complejos se descartó completamente para el proyecto.

5.8. Claude (Anthropic)

El desarrollo de motores de ajedrez es un área mayormente de código abierto, lo que tiene ventajas claras pero también limitaciones prácticas. La documentación disponible tiende a ser densa, incompleta o escrita asumiendo un conocimiento previo significativo del dominio. La Chess Programming Wiki es la referencia más completa del área, pero muchos artículos son escasos, están desactualizados o simplemente enlazan a recursos que ya no existen. La documentación oficial de proyectos como Stockfish o Bullet asume que el lector ya entiende los fundamentos, y los foros y repositorios donde se discuten técnicas avanzadas están dispersos y no siempre accesibles.

En este contexto, Claude (Anthropic) se utilizó como complemento a las fuentes primarias, no como sustituto de ellas. El flujo de trabajo típico consistía en leer la documentación disponible, identificar los puntos donde la comprensión era incompleta, y formular una consulta que pusiera a prueba o completara ese entendimiento. Un ejemplo concreto: al leer un ciclo de entrenamiento histórico de Stockfish [13], el documento describe decisiones de hiperparámetros y selección de candidatos sin explicar el razonamiento detrás de cada una. Consultar si la interpretación de ese proceso era correcta, y pedirle que señalara qué partes eran generalizables y cuáles específicas de la escala de Stockfish, permitió extraer conclusiones aplicables al proyecto sin sobreinterpretar el ejemplo original.

Este patrón se repitió a lo largo del proyecto: leer la fuente, formular una hipótesis sobre lo que significaba, y usar el modelo para validarla o corregirla. Esto fue especialmente valioso en el dominio de los motores de ajedrez, donde términos como *zugzwang*, SEE o LOS aparecen sin definición en la mayoría de fuentes y donde la diferencia entre una implementación correcta y una sutilmente incorrecta de una heurística puede ser difícil de detectar sin un punto de referencia externo.

A continuación se presentan prompts representativos de las distintas etapas del proyecto, junto con una breve nota sobre qué aportó cada interacción.

Entendimiento del codebase base

I'm building a chess engine in Rust by adapting an existing open source engine called Reckless. I have no prior experience with this codebase and I need to understand its structure before modifying it. Can you walk me through how the board representation, move generation, and search are organized? I want to keep the infrastructure (bitboards, UCI, transposition table) but strip out the neural network evaluation and replace it with my own. What should I look for and what's safe to remove?

Este fue el primer punto de contacto con el proyecto. *Reckless* es un motor optimizado para rendimiento y su estructura no es obvia para alguien sin experiencia previa en el dominio. La respuesta permitió identificar rápidamente qué módulos eran reutilizables y cuáles debían modificarse, reduciendo el tiempo de orientación inicial de días a horas. Con ese mapa mental, releer el código fuente directamente fue un proceso mucho más directo.

Heurísticas de búsqueda a implementar

I'm building a chess engine in Rust. I have a working alpha-beta search with iterative deepening and quiescence search. I want to add search heuristics incrementally to measure the ELO impact of each one. What are the most impactful pruning and reduction techniques I should implement first? For each one, can you explain the intuition behind it, roughly where in the search it applies, and any conditions where it should be disabled?

La Chess Programming Wiki lista decenas de técnicas sin indicar cuáles son fundamentales y cuáles son optimizaciones marginales. Esta consulta produjo una lista priorizada con intuición para cada técnica, que luego se usó como guía para decidir el orden de implementación. Sirvió como índice para saber qué buscar en la documentación formal, no como sustituto de ella.

Implementación de heurísticas específicas

I'm adding Late Move Reductions to my Rust chess engine. My search is a standard negamax with alpha-beta and move ordering (TT move first, then captures sorted by SEE, then quiet moves by history). Where exactly should I apply the reduction, what formula should I use for the reduction depth, and under what conditions should I skip it entirely? Are there any interactions with other heuristics I already have like null move pruning that I need to be careful about?

Las heurísticas de búsqueda tienen condiciones de activación que interactúan entre sí de formas no siempre documentadas explícitamente. Proporcionar el estado actual del motor como contexto permitió obtener respuestas específicas a la implementación existente, no genéricas. Esto fue útil para evitar errores comunes como aplicar LMR dentro de la búsqueda de quietud o en nodos donde ya se aplicó null move.

Arquitectura y entrenamiento de NNUEs

I want to add NNUE evaluation to my chess engine. I understand the basic idea of efficiently updatable networks but I need help choosing the right architecture and training setup for a first implementation. I'm using a 768-feature input (12 piece types x 64 squares), and I plan to use the Bullet trainer. What hidden layer size should I start with, what activation function is standard, and how does the perspective network work? Also, what does a typical training run look like in terms of epochs, learning rate schedule and when to stop?

La documentación de NNUE asume familiaridad con redes neuronales cuantizadas y con conceptos específicos del dominio como el acumulador incremental. Esta consulta permitió construir un modelo mental de la arquitectura antes de leer la documentación técnica de Stockfish [10] y el código de Bullet [11], que de otro modo habrían sido difíciles de seguir sin ese contexto previo.

Selección de checkpoints y experimentos con fastchess

I'm running experiments comparing different NNUE training checkpoints using fastchess. I don't have enough compute to run full gauntlets for every candidate, so I want to compare checkpoints at different stages: early epochs, loss plateau, and final epoch. How should I interpret the fastchess output (ELO, nELO, LOS, draw ratio)? What does LOS tell me about statistical significance, and how many games do I need for the results to be meaningful? Is comparing checkpoints head to head a valid substitute for running each against a fixed reference?

La interpretación de resultados de torneo no es intuitiva: ELO, nELO y LOS miden cosas distintas y la diferencia importa al tomar decisiones sobre qué candidato promover. Esta consulta ayudó a entender qué significan estadísticamente los resultados de fastchess y a diseñar una estrategia de experimentación razonable dentro de las restricciones de cómputo disponibles.

6. Resultados

6.1. Comparación entre versiones

Todos los torneos se ejecutaron con **fastchess** [3] bajo condiciones homogéneas: control de tiempo 10+0,1s, 1 hilo, 16 MB de hash, 600 partidas y *opening book* `UH0_Lichess.epd`.

Cuadro 1: Resultados head-to-head entre versiones consecutivas (600 partidas, 10+0,1s, `UH0_Lichess.epd`).

Retador	Rival	ELO	nELO	LOS	W / D / L
v1.5-move-ordering	v1-base	+321 ± 30	+498	100 %	465 / 100 / 31
v2-tapered-eval	v1.5-move-ordering	+235 ± 32	+275	100 %	436 / 82 / 82
v2.4-additional-heuristics	v2-tapered-eval	+564 ± 59	+1089	100 %	561 / 33 / 6
v3-nnue	v2.4	+545 ± 61	+948	100 %	561 / 28 / 11
v4-nnue-heuristics	v3-nnue	+71 ± 18	+115	100 %	247 / 227 / 126

Cuadro 2: Resultados contra Stockfish a nivel fijo (600 partidas, 10+0,1s, `UH0_Lichess.epd`).

Versión	SF nivel	ELO	LOS	W / D / L	Puntos
v1-base	2000	+55 ± 27	100 %	337 / 21 / 242	347.5 (57.9 %)
v1.5-move-ordering	2500	+14 ± 24	86,9 %	293 / 38 / 269	312.0 (52.0 %)
v2.4-additional-heuristics	2800	-62 ± 24	0 %	205 / 84 / 311	247.0 (41.2 %)
v3-nnue	3190	+8 ± 21	76,3 %	226 / 161 / 213	306.5 (51.1 %)
v4-nnue-heuristics	3190	+59 ± 23	100 %	275 / 151 / 174	350.5 (58.4 %)

6.2. Análisis de resultados

6.2.1. Ordenamiento de movimientos

Agregar ordenamiento de movimientos produce un salto de +321 ELO sobre **v1-base**. Alpha-beta sin ordenamiento explora candidatos en un orden arbitrario, lo que limita severamente cuánto del

árbol puede podarse. Al priorizar capturas, movimientos de la tabla de transposiciones y candidatos con buen historial, el motor ve buenas jugadas antes y genera cortes beta más temprano. El resultado es que se explora un árbol de búsqueda mucho más pequeño a la misma profundidad, o una profundidad mayor en el mismo tiempo.

6.2.2. Evaluación graduada y heurísticas adicionales

La evaluación graduada (**v2-tapered-eval**) introduce conciencia de fase de juego. Un caballo centralizado vale mucho en el *midgame* pero poco en un final de peones, mientras que un rey activo es una debilidad en el *midgame* pero esencial en el final. Al mantener dos tablas separadas e interpolar según el material restante, el motor empieza a tomar decisiones distintas según la fase.

Las técnicas acumuladas en **v2.4** producen el mayor salto entre versiones HCE consecutivas (+564 ELO sobre **v2-tapered-eval**). Las extensiones singulares son la contribución más notable, en posiciones donde existe una única respuesta que evita un deterioro significativo, el motor la identifica y la analiza a mayor profundidad sin incrementar el costo de búsqueda de forma proporcional. Es decir, **v2.4** explora un poco más allá en las posiciones prometedoras por lo que al enfrentarse a una iteración anterior que no lo hace logra ver movimientos que el otro motor ignora. Esto explica el *nELO* de +1000 a pesar que no sea un motor con capacidades tan por encima de su predecesor. LMR y RFP complementan esto reduciendo el tiempo invertido en movimientos que probablemente no cambien el resultado.

6.2.3. Transición a NNUE

El salto de **v2.4** a **v3-*nnue*** es el más grande del proyecto en términos de ELO absoluto (≈ 460 puntos de diferencia entre estimaciones, utilizando Stockfish. Las estimaciones entre diferentes iteraciones del mismo motor no son necesariamente indicativas del ELO real). Hay dos razones principales para esto.

La primera es que la HCE implementada es intencionalmente simple. Captura balance material y posicionamiento básico, pero los motores HCE más potentes incluyen cientos de términos adicionales como seguridad del rey, movilidad de piezas, estructura de peones, par de alfiles ajustados durante años por expertos del dominio. La NNUE aprende representaciones equivalentes y más complejas directamente de los datos, sin requerir que ese conocimiento sea codificado manualmente.

La segunda razón es que una evaluación más precisa amplifica el efecto de todas las heurísticas de búsqueda existentes. Podas como RFP o NMP asumen que la evaluación estática es una aproximación razonable del valor real de la posición. Cuando esa aproximación mejora, las podas se vuelven más seguras y el ordenamiento de movimientos más efectivo. Las heurísticas implementadas, aunque diseñadas bajo supuestos de HCE, transfieren bien a una evaluación neuronal porque sus principios son independientes del tipo de evaluador.

6.2.4. Refinamiento de búsqueda sobre NNUE

v4-*nnue-*heuristics** añade +71 ELO sobre **v3-*nnue*** en el enfrentamiento directo. La ganancia es más modesta en términos relativos, pero es significativa considerando que la evaluación no cambia. Toda la mejora proviene exclusivamente de usar mejor el tiempo de búsqueda disponible.

Un aspecto relevante de **v4** es que parte de su mejora proviene de reajustar parámetros de poda diseñados originalmente bajo HCE. Una evaluación neuronal es más estable y consistente que una HCE, produce menos oscilaciones en la puntuación y sus estimaciones son más cercanas al

resultado real. Esto permite utilizar márgenes de poda más ajustados sin aumentar el riesgo de descartar posiciones buenas, y técnicas como la heurística de mejora pueden tomar decisiones más informadas sobre cuándo ser conservador y cuándo podar agresivamente.

Las nuevas técnicas de búsqueda (ventanas de aspiración, razoring, IIR, extensiones de jaque, poda de futilidad y poda SEE en búsqueda) actúan en distintas capas del árbol de búsqueda y se complementan. Las ventanas de aspiración reducen el trabajo en profundidades altas cuando la posición es estable, mientras que razoring y futility pruning recortan nodos superficiales claramente perdedores. El resultado neto es que el motor dedica más nodos a las partes del árbol que realmente importan.

6.2.5. Contexto de los ELO estimados

Como prefacio a la sección, es importante destacar que los valores de ELO no son una conversión directa entre humanos y motores. Es de esperarse que un humano pueda ganarle partidas a motores con un ELO más alto que ellos, en especial considerando algunas de las implementaciones básicas y las debilidades con las que cuentan. Se entra a más detalle en la siguiente sección del reporte.

Cuadro 3: ELO absoluto estimado para versiones con referencia directa contra Stockfish.

Versión	ELO estimado	Equivalencia aproximada
v1-base	≈ 2055	Maestro FIDE
v1.5-move-ordering	≈ 2514	Gran Maestro Internacional
v2.4-additional-heuristics	≈ 2738	Élite humana / top mundial
v3- <code>nnue</code>	≈ 3198	Sobrehumano
v4- <code>nnue-heuristics</code>	≈ 3249	Sobrehumano

v1-base alcanza ≈ 2055 ELO, equivalente a un Maestro FIDE. Es un punto de partida razonablemente fuerte considerando que no tiene ordenamiento de movimientos, y refleja en parte la solidez de la infraestructura base: tener bitboards, tabla de transposiciones y búsqueda de quietud ya evita errores tácticos gruesos.

v1.5 llega a ≈ 2514 ELO, nivel de Gran Maestro Internacional. Que únicamente el ordenamiento de movimientos produzca ese salto desde 2055 ilustra cuánto potencial desperdicia alpha-beta sin una buena estrategia de exploración.

v2.4 alcanza ≈ 2738 ELO, un nivel que corresponde a los mejores jugadores humanos del mundo. Es un resultado notable para una HCE relativamente simple y sugiere que con términos adicionales de evaluación, esta familia de técnicas podría acercarse al nivel del Stockfish pre-NNUE (≈ 3000).

v3-`nnue` llega a ≈ 3198 ELO, muy por encima del pico histórico de Magnus Carlsen (≈ 2882 ELO). A este nivel el motor juega de forma consistentemente sobrehumana y la brecha respecto a **v2.4** refleja directamente el techo de la evaluación heurística primitiva.

v4-`nnue-heuristics` alcanza ≈ 3249 ELO, una mejora adicional de 51 puntos sobre **v3** sin cambiar la red neuronal. Demuestra que incluso con una evaluación de calidad, los refinamientos de búsqueda siguen aportando ganancias concretas.

6.3. Partidas contra oponente humano

Se realizaron partidas de prueba contra un voluntario con ELO estimado de ≈ 2200 para analizar cualitativamente el comportamiento de las versiones principales del motor desde una perspectiva humana. Las observaciones complementan los resultados de torneo al revelar tendencias que los números de ELO no capturan directamente.

6.3.1. v1.5-move-ordering

El jugador fue capaz de ganar las partidas contra v1.5, pero la estrategia requirió adaptación. En posiciones de mediojuego el motor jugaba de forma coherente y ejercía cierta presión posicional, lo que dificultaba ganar material de forma directa. La debilidad apareció de forma consistente al transicionar hacia el final: sin evaluación graduada, v1.5 aplica los mismos criterios posicionales en todas las fases del juego, lo que produce decisiones claramente subóptimas en finales donde las piezas cambian de rol. Un jugador con comprensión de finales puede explotar esto al simplificar la posición intercambiando piezas, llevar el juego a una estructura de peones donde el motor pierde el hilo, y convertir desde ahí. En la práctica, un humano competente con paciencia posicional tiene una ruta clara para ganar contra esta versión.

6.3.2. v2.4-additional-heuristics

Contra v2.4 la experiencia fue bastante distinta. En la primera partida el voluntario logró tablas, pero el proceso fue notablemente más difícil. El motor mantuvo presión posicional constante, forzando al jugador a calcular con más cuidado y conceder poco espacio. La segunda partida fue más concluyente, el motor adoptó un estilo abiertamente agresivo y fue sufocando progresivamente al rival sin darle tiempo de reorganizarse.

La diferencia de comportamiento respecto a v1.5 es atribuible principalmente a las heurísticas acumuladas en v2.4, en particular las extensiones singulares. Al buscar más profundo en líneas tácticas críticas, el motor detecta amenazas y recursos que versiones anteriores dejaban fuera del horizonte de búsqueda. El resultado es que v2.4 juega posiciones complejas de forma más agresiva y menos pasiva, no evita complicaciones si no que las busca cuando la evaluación le indica que tiene ventaja táctica. Esto coincide con la observación del jugador de que el motor “se volvió más agresivo”. No es un cambio de estilo completamente, sino la consecuencia de que el motor ahora puede respaldar sus decisiones con más profundidad de cálculo en las posiciones que importan.

7. Conclusión

El desarrollo incremental del motor confirmó que cada una de las técnicas estudiadas tiene un impacto medible y justificable en la fuerza de juego. Más que un conjunto de optimizaciones independientes, el resultado es un sistema donde cada componente potencia a los demás.

7.1. Metodología incremental

Construir el motor como una progresión de versiones independientes resultó ser una forma efectiva de estudiar el dominio. Al aislar cada técnica en su propia rama, fue posible atribuir diferencias de ELO a causas concretas en lugar de especular sobre qué cambio produjo qué efecto. Esto también facilitó identificar interacciones entre técnicas, el ordenamiento de movimientos por ejemplo, amplifica el impacto de la evaluación porque una evaluación más precisa se traduce directamente

en mejores decisiones de poda. La metodología tiene limitaciones reales en presupuesto de cómputo, pero dentro de esas restricciones, comparar versiones directamente contra Stockfish a nivel fijo resultó ser una métrica de referencia confiable y reproducible.

7.2. Evaluación heurística

Una HCE basada únicamente en tablas pieza-casilla alcanzando 2055 ELO es un resultado más sólido de lo que podría parecer a primera vista. Las PST codifican principios posicionales fundamentales (centralización, actividad de piezas, seguridad del rey en el mediojuego) de forma extremadamente compacta, y la infraestructura de búsqueda se encarga del resto. La evaluación graduada PeSTO [16] demostró que añadir conciencia de fase de juego produce un salto considerable sin complejidad de implementación desproporcionada.

Lo interesante de la HCE es el rango que deja abierto. Con solo PSTs el motor llega a 2055 ELO; el Stockfish pre-NNUE, con una HCE desarrollada durante años por expertos, alcanzaba aproximadamente 3000. Esa brecha representa todo el conocimiento ajedrecístico que puede codificarse explícitamente. Por ejemplo, movilidad de piezas, seguridad del rey, estructura de peones, par de alfiles, torres en columnas abiertas, y decenas de términos adicionales. Explorar aunque sea una fracción de ese espacio sería un trabajo futuro interesante dentro de este proyecto.

7.3. Ordenamiento de movimientos y heurísticas de búsqueda

El ordenamiento de movimientos resultó ser uno de los cambios más influyentes del proyecto. Alfabeta sin ordenamiento es fundamentalmente ineficiente, la misma búsqueda con buen ordenamiento puede alcanzar profundidades significativamente mayores en el mismo tiempo. Este efecto se amplifica con evaluaciones más precisas, ya que una mejor evaluación produce un mejor ordenamiento, que a su vez permite podas más agresivas.

Las heurísticas de poda son igualmente críticas en un juego con el factor de ramificación del ajedrez. Sin ellas el árbol de búsqueda crece de forma inmanejable, mientras que con ellas mal calibradas el motor descarta líneas importantes. El balance entre podar agresivamente y no ignorar movimientos clave es uno de los problemas más interesantes del dominio, y v4 ilustra que ese balance depende también del tipo de evaluador. Los parámetros óptimos bajo HCE no lo son necesariamente bajo NNUE.

7.4. NNUE

La transición a evaluación neuronal produjo el mayor salto absoluto de ELO del proyecto. Incluso con una arquitectura modesta y un dataset de tamaño razonable, la NNUE supera ampliamente a la HCE manual. Esto refleja la dificultad de codificar conocimiento ajedrecístico de forma explícita, los patrones que una red aprende automáticamente de los datos requieren años de trabajo experto para capturarse en una HCE.

El proceso de entrenamiento es fundamentalmente empírico. No existe una fórmula que determine de antemano qué arquitectura, qué dataset o qué protocolo de ajuste fino producirá el mejor resultado. Los experimentos realizados ilustran esto con claridad, los protocolos que mejoraron una arquitectura perjudicaron a otra, y las diferencias entre combinaciones de datos fueron menores de lo esperado. Es un proceso que requiere cómputo considerable y tolerancia a resultados inesperados. Al mismo tiempo, la comunidad de motores de código abierto comparte activamente hallazgos,

datos y herramientas, lo que reduce considerablemente la barrera de entrada para experimentar y aprender.

7.5. Trabajo futuro

Varias líneas de trabajo quedan abiertas a partir de este proyecto.

En cuanto a la evaluación, el siguiente paso natural sería implementar una HCE más completa como línea base, y explorar arquitecturas NNUE con *feature sets* más expresivos como HalfKP o HalfKA, que relativizan las características a la posición del rey y capturan relaciones espaciales que el feature set de 768 entradas no puede representar.

En cuanto a infraestructura, uno de los cuellos de botella del proyecto fue la falta de un pipeline de evaluación automatizado. Stockfish resuelve esto enviando redes candidatas automáticamente a un sistema de torneos distribuido que corre continuamente. Replicar algo similar a menor escala, incluso corriendo gauntlets automatizados durante la noche o un fin de semana, permitiría explorar el espacio de hiperparámetros y arquitecturas de forma mucho más sistemática sin intervención manual por cada experimento.

Finalmente, un objetivo personal es reescribir el motor íntegramente desde cero en Rust, sin partir de la infraestructura de *Reckless*. Este proyecto reutilizó la representación de tablero, la generación de movimientos y el protocolo UCI como punto de partida, lo que fue valioso para enfocarse en búsqueda y evaluación. Pero construir esos componentes desde cero implicaría un entendimiento más profundo de cada capa del motor y daría control total sobre decisiones de diseño que actualmente están heredadas.

Referencias

- [1] codedeliveryservice. Reckless chess engine. <https://github.com/codedeliveryservice/Reckless>, 2024. GitHub repository.
- [2] Stockfish Contributors. Stockfish chess engine. <https://github.com/official-stockfish/Stockfish>, 2024. GitHub repository.
- [3] Disservin and contributors. fastchess. <https://github.com/Disservin/fastchess>, 2024. GitHub repository.
- [4] En-Croissant Contributors. En-croissant chess GUI. <https://encroissant.org>, 2024. Open-source chess interface.
- [5] Stockfish Contributors. nnue-pytorch: NNUE trainer for stockfish. <https://github.com/official-stockfish/nnue-pytorch>, 2024. GitHub repository.
- [6] jw1912. Bullet NNUE trainer. <https://github.com/jw1912/bullet>, 2024. GitHub repository.
- [7] José Antonio Mérida Castejón. Chess engine. <https://github.com/TonitoMC/chess-engine>, 2026. GitHub repository.
- [8] Chess Programming Wiki. Evaluation. <https://www.chessprogramming.org/Evaluation>, 2024. Chess Programming Wiki.
- [9] Chess Programming Wiki. NNUE. <https://www.chessprogramming.org/NNUE>, 2024. Chess Programming Wiki.
- [10] Stockfish Contributors. NNUE documentation — nnue-pytorch. <https://github.com/official-stockfish/nnue-pytorch/blob/master/docs/nnue.md>, 2024. GitHub repository.
- [11] jw1912. Bullet NNUE trainer documentation. <https://github.com/jw1912/bullet/blob/main/docs/0-contents.md>, 2024. GitHub repository.
- [12] Stockfish Contributors. Training datasets — nnue-pytorch wiki. <https://github.com/official-stockfish/nnue-pytorch/wiki/Training-datasets>, 2024. GitHub Wiki.
- [13] Stockfish Contributors. Training examples (historical) — nnue-pytorch wiki. <https://github.com/official-stockfish/nnue-pytorch/wiki/Training-examples>, 2024. GitHub Wiki.
- [14] Linrock. test80-mar2023 / 2tb7p-v6-sk16 — Leela Chess Zero training data. <https://www.kaggle.com/datasets/linrock/test80-mar2023-2tb7p-v6-sk16>, 2023. Kaggle dataset.
- [15] Chess Programming Wiki. Simplified evaluation function. https://www.chessprogramming.org/Simplified_Evaluation_Function, 2024. Chess Programming Wiki.
- [16] Chess Programming Wiki. PeSTO's evaluation function. [https://www.chessprogramming.org/PeSTO's_Evaluation_Function](https://www.chessprogramming.org/PeSTO%27s_Evaluation_Function), 2024. Chess Programming Wiki.
- [17] Chess Programming Wiki. Null move pruning. https://www.chessprogramming.org/Null_Move_Pruning, 2024. Chess Programming Wiki.
- [18] Chess Programming Wiki. Late move reductions. https://www.chessprogramming.org/Late_Move_Reductions, 2024. Chess Programming Wiki.

- [19] Chess Programming Wiki. Reverse futility pruning. https://www.chessprogramming.org/Reverse_Futility_Pruning, 2024. Chess Programming Wiki.